

Laboratori DAT / 1

Functors, Applicatives i Monads

Índex

1	Introducció	2
2	Màquina de pila simple	2
2.1	Tractament d'errors	2
2.1.1	Tractament d'errors amb el <code>Monad Maybe</code>	2
2.1.2	Tractament d'errors amb el <code>Monad (Either String)</code>	2
2.2	Modificació de l'estat amb monads	2
3	Analitzadors	3
3.1	Definició del tipus <code>Analitzador</code>	3
3.2	Analitzadors bàsics	3
3.3	Com combinar analitzadors ?	4
3.4	<code>Alternative</code>	5
4	Llenguatge d'expressions aritmètiques	6
5	Recordatori de funcions	8
6	Llistats	9
6.1	Màquina de pila simple (versió 1)	9
6.2	Màquina de pila simple (versió 2)	10
6.3	Màquina de pila simple (versió 3)	11
6.4	Màquina de pila simple amb monad estat	12
6.5	Màquina d'expressions	13

1 Introducció

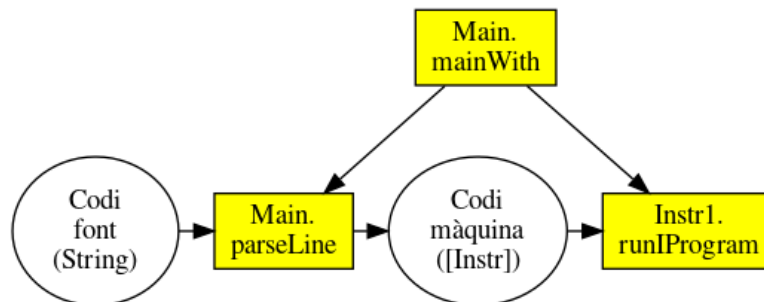
En aquesta pràctica es proposa un projecte on apareixen functors, aplicatius i monads. Es vol realitzar un simple avaluador d'expressions aritmètiques amb números enters.

2 Màquina de pila simple

Es tracta d'una màquina de pila molt simple que llegeix i executa un programa (de baix nivell) per aquesta màquina de pila.

La realització de la primera versió d'aquesta senzilla màquina de pila està formada pels mòduls **Main** (fitxer [mps1.hs](#)), **Instr1** i **Ops**. Els mòduls **Instr1** i **Ops** defineixen el tipus de les instruccions de la màquina i l'execució d'un programa (llista d'instruccions).

El mòdul **Main** defineix com llegir un fitxer amb el text del programa, com convertir-lo a llista d'instruccions, i finalment invocar la seva execució i mostrar el resultat.



Podeu provar aquesta màquina amb el següent programa:

```
3 5 Mul 7 Div 2 1 Sub Mul
```

i observeu que obté el resultat final 2.

2.1 Tractament d'errors

2.1.1 Tractament d'errors amb el Monad Maybe

En una segona versió, es tracten els errors o situacions anòmales amb el Monad **Maybe**. Aquesta segona versió està formada pels mòduls **Main** (fitxer [mps2.hs](#)), **Instr2** i **Ops**.

Exercici:

Completeu la funció `traverseList` del fitxer [mps2.hs](#) recursivament i usant només funcions de **Functor** i d'**Applicative**.

2.1.2 Tractament d'errors amb el Monad (Either String)

Exercici:

Completeu els fitxers [mps3.hs](#) i [Instr3.hs](#) corresponents a una nova versió en que es tracten els errors amb el Monad **(Either String)**.

2.2 Modificació de l'estat amb monads

En aquest apartat introduïrem un monad que s'encarregui de mantenir un estat intern. Per això, modificarem la realització de la màquina de pila de la primera versió (sense tractament dels errors) amb un mòdul **Instr1Estat** que oferirà el mateix API que el mòdul **Instr1**, però definint i usant el monad **ICodeM** (veieu el codi).

Exercici:

Completeu el fitxer [Instr1Estat.hs](#).

Modifiqueu el fitxer [mps1.hs](#) per a que importi el mòdul **Instr1Estat** en lloc del **Instr1** i proveu la vostra realització.

3 Analitzadors

En aquesta secció introduïrem els analitzadors que després, en la següent secció, usarem en la realització de la màquina que tractarà un llenguatge d'expressions de més alt nivell.

Un analitzador és un algorisme que té com a entrada dades no estructurades (per exemple un `String`) i dóna com a resultat un valor d'un tipus de dades estructurat, definit per l'aplicació i que permeti el posterior tractament de manera adequada.

3.1 Definició del tipus Analitzador

Anem a definir un analitzador per a una entrada de tipus `String` (llista de `Chars`).

Per nosaltres, un analitzador de tipus `Analitzador a` (per a un de tipus `a` qualsevol), és bàsicament una funció que té com a paràmetre l'entrada a analitzar (de tipus `String`), i té èxit o fracassa. Si té èxit, retorna el valor analitzat juntament amb la part de l'entrada que no ha utilitzat:

```
newtype Analitzador a = Analitzador (String -> Maybe (a, String))

execAnalitzador :: Analitzador a -> String -> Maybe (a, String)
execAnalitzador (Analitzador f) s = f s
```

3.2 Analitzadors bàsics

Caràcters

Com a exemple anem a veure un analitzador pel tipus `Char`.

La funció `complirA` té com a parametre una funcio que representa un predicat sobre un `Char` i té com a resultat un analitzador. Aquest analitzador:

- té èxit només si veu un `Char` que compleix el predicat (que alhora és el `Char` retornat per l'analitzador).
- si troba un caràcter que no compleix el predicat (o l'entrada és buida) llavors falla i la sortida és `Nothing`.

```
complirA :: (Char -> Bool) -> Analitzador Char
complirA predicat = Analitzador f
  where
    f [] = Nothing      -- falla si l'entrada es buida
    f (x:xs) =          -- Comprova si x compleix el predicat
      if predicat x then Just (x, xs) -- si es que si
                                -- retorna x i la resta de
                                -- l'entrada (es a dir, xs)
      else Nothing      -- altrament, falla
```

Mijançant `complirA`, es pot definir un analitzador que espera trobar exactament el caràcter que es passa a l'entrada i si no falla.

```
caracterA :: Char -> Analitzador Char
caracterA c = complirA (== c)
```

Exercici:

Avalueu i enteneu el resultat de les següents expressions:

```
execAnalitzador (complirA isUpper) "ABC"
execAnalitzador (complirA isUpper) "abc"
execAnalitzador (caracterA 'x') "xyz"
```

Enters

Observeu el següent analitzador. Si l'entrada comença per caràcters numèrics els retorna tots fins que troba un caràcter no numèric.

```
enterPosA :: Analitzador Int
enterPosA = Analitzador f
  where
    f xs = if null digits then Nothing
           else Just (read digits, elQueQueda)
           where (digits, elQueQueda) = span isDigit xs
```

Exercici:

Proveu l'analitzador per diferents entrades, per exemple:

```
ghci> execAnalitzador enterPosA "234abc"
```

3.3 Com combinar analitzadors ?

Implementar analitzadors de manera explícita és llarg i és fàcil cometre errors per a qualsevol realització que vagi més enllà dels analitzadors primitius més bàsics. La idea és aconseguir tenir la capacitat de crear analitzadors complexos a partir de la combinació d'altres de més simples. Aquesta capacitat ens la donarà la classe **Applicative**.

Exercici:

Implementeu la instància de **Functor** per **Analitzador**.

Implementeu la instància d'**Applicative** per **Analitzador**. Cal tenir en compte que:

- **pure x** representa l'analitzador que no consumeix cap entrada i retorna amb èxit un resultat **x**.
- **f1 <*> f2** representa l'analitzador que executa primer **f1** (que consumeix part de l'entrada i produeix una funció), després passa la part de l'entrada que no ha consumit a **f2** (que consumeix més entrada i produeix algun valor), i finalment retorna el resultat d'aplicar la funció al valor.
Si **f1** o **f2** fallen, tot l'analitzador també falla (dit d'una altra manera, **f1 <*> f2** només té èxit si tant **f1** com **f2** tenen èxit).

Exemple:

```
analitzaParella :: Analitzador (Char, Int)
analitzaParella = (,) <$> complirA isUpper <*> enterPosA
```

amb aquest analitzador podem veure els següents resultats:

```
ghci> execAnalitzador analitzaParella "F345xzf"
Just (('F', 345), "xzf")
```

```
ghci> execAnalitzador analitzaParella "345F"
Nothing
```

Exercici:

Tenint en compte que **Analitzador** és una instància d'**Applicative**, realitzeu un analitzador **cadenaA :: String -> Analitzador String** que, donat un **String** (llista de caràcters), espera trobar exactament els caràcters indicats i els retorna. Podeu usar recursivitat sobre la llista de caràcters.

3.4 Alternative

Com s'ha vist `Applicative` es pot utilitzar per fer analitzadors per a formats senzills i fixos. Però que passa quan un format implica triar (per exemple, “paraules que comencen per vocal”)? En aquest cas `Applicative` no és suficient. Per poder triar existeix la classe `Alternative` que està definida com,

```
class Applicative f => Alternative f where
  empty :: f a
  (<|>) :: f a -> f a -> f a
  some :: f a -> f [a]
  more :: f a -> f [a]
```

Exercici:

Implementeu la instància d'`Alternative` per `Analitzador`. Cal tenir en compte que:

- `empty`: és l'analitzador que sempre falla.
- `f1 <|> f2`: és un analitzador que primer executa `f1`. Si `f1` té èxit llavors ignora `f2` i el resultat és el resultat de `f1`. Però si `f1` falla, llavors executa `f2` i aquest serà el resultat.

Exercici:

Feu un analitzador `aeiouA :: Analitzador Char` que el primer que espera trobar és una vocal.

Exercici:

Feu un analitzador `vocNum :: Analitzador (Either Char Int)` que el primer que espera trobar és una vocal o un enter positiu, usant `aeiouA` i `enterPosA`. Observeu el tipus de la funció.

Moltes vegades serà molt convenient usar dues funcions definides en la classe `Alternative` (tenen implementacions per defecte):

```
many :: Alternative f => f a -> f [a]
some :: Alternative f => f a -> f [a]
```

- `many`: que té com a paràmetre un analitzador i l'executa de manera consecutiva tantes vegades com sigui possible (que podria ser cap, si falla el primer cop), retornant una llista dels resultats. Observeu que `many` sempre té èxit executant zero o més vegades l'analitzador.
- `some`: és similar, excepte que requereix que l'analitzador indicat com a paràmetre tingui èxit almenys una vegada. Si l'analitzador indicat falla immediatament `some` també falla.

Exemples d'ús:

```
ghci> execAnalitzador (many (complirA isDigit)) "123abc"
Just ("123","abc")
ghci> execAnalitzador (some (complirA isDigit)) "123abc"
Just ("123","abc")
```

Observeu la diferència en el següent cas:

```
ghci> execAnalitzador (many (complirA isDigit)) "abc123"
Just ("","abc123")
ghci> execAnalitzador (some (complirA isDigit)) "abc123"
Nothing
```

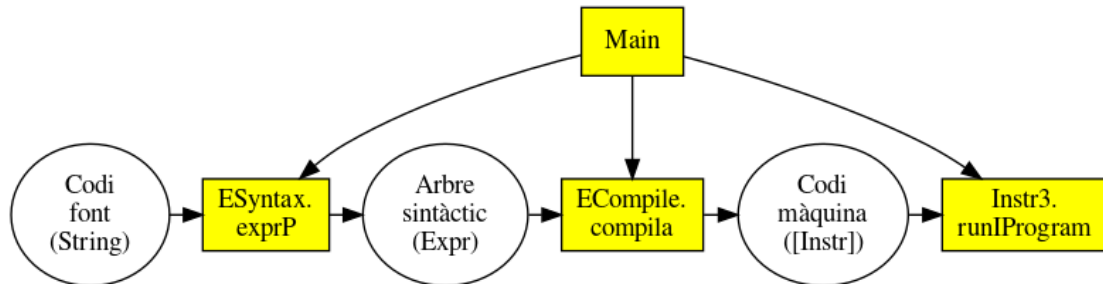
Exercici:

Realitzeu una nova implementació de l'analitzador `enterPosA :: Analitzador Int`, a partir de l'analitzador `complirA isDigit` i usant la funció `some`.

4 Llenguatge d'expressions aritmètiques

Definim una màquina avaluadora d'expressions una mica de més alt nivell, que s'encarregui d'analitzar la sintaxi d'una expressió (en notació prefix i a l'estil curried de Haskell), analitzar la validesa semàntica i finalment generar les instruccions de la màquina de pila i executar-les.

La realització de la màquina avaluadora d'expressions està formada pels mòduls `Main` (fitxer `me.hs`), `ESyntax`, `Analitzador`, `ECompile`, `Instr3` i `Ops`.



L'entrada de l'analitzador serà un text del llenguatge d'expressions, i el resultat serà el corresponent arbre sintàctic de tipus `Expr` (veieu el mòdul `ESyntax`). La sintaxi d'aquest llenguatge d'expressions ve donada per les següents regles gramaticals:

- Una expressió és
 - un àtom, o bé,
 - un identificador seguit d'un o més àtoms.
- Un àtom és
 - un número, o bé,
 - una expressió entre parèntesis.

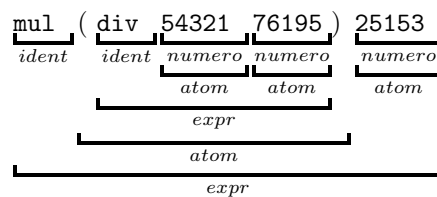
En [notació de Backus-Naur](#):

```

expr = atom
      | ident atom+
atom = numero
      | "(" expr ")"
  
```

que haureu de realitzar amb els corresponents analitzadors `exprP` i `atomP`.

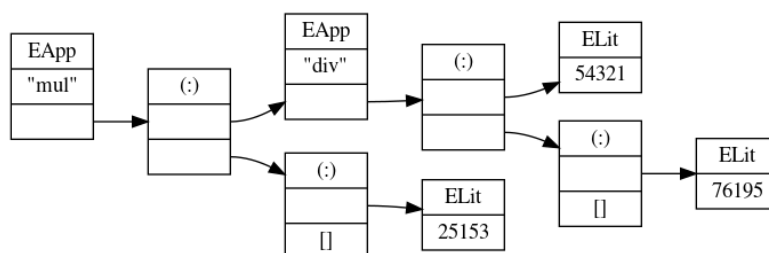
Per exemple, donada la següent expressió:



un cop analitzada, hauriem d'obtenir el següent valor de tipus `Expr`:

```

EApp "mul" [EApp "div" [ELit 54321, ELit 76195], ELit 25153]
  
```



Posteriorment, el resultat de tipus **Expr** serà validat per la funció `compile` del mòdul [ECompile](#). Aquesta funció, en cas d'èxit, obté la llista d'instruccions corresponent que haurà d'executar la màquina de pila.

Podeu provar aquest avaluador amb la següent expressió:

```
mul (div (mul 3 5) 7) (sub 2 1)
```

i observeu que obté el resultat final 2.

Exercici:

Completeu el mòdul [ESyntax](#).

Exercici:

Completeu el mòdul [ECompile](#).

5 Recordatori de funcions

```
fmap :: Functor f => (a -> b) -> f a -> f b

(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap

(<$) :: Functor f => a -> f b -> f a
(<$) = fmap . const

pure :: Applicative f => a -> f a

(<*>) :: Applicative f => f (a -> b) -> f a -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c

(*>) :: Applicative f => f a -> f b -> f b
(*>) = liftA2 (flip const)

(<*) :: Applicative f => f a -> f b -> f a
(<*) = liftA2 const

traverse_ :: (Foldable t, Applicative f) => (a -> f b) -> t a -> f ()

traverse :: (Traversable t, Applicative f) => (a -> f b) -> t a -> f (t b)

empty :: Alternative f => f a

(<|>) :: Alternative f => f a -> f a -> f a

some :: Alternative f => f a -> f [a]

many :: Alternative f => f a -> f [a]

(>=) :: Monad m => m a -> (a -> m b) -> m b

foldlM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b

mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()

mapM :: (Traversable t, Monad m) => (a -> m b) -> t a -> m (t b)
```


6 Llistats

6.1 Màquina de pila simple (versió 1)

Llistat 1: mps1.hs

```
-- Us de la funcio 'fmap' que mapeja una funcio a un contenidor
-- o 'Functor'. Les funcions 'fmap' o '<$>' son mes generals
-- que la funcio 'map' i no nomes s'apliquen a llistes.

module Main where
import Instr1
import Ops

import System.Environment (getArgs)

main :: IO ()
main = do
    [filename] <- getArgs
    mainWith filename

mainWith :: String -> IO ()
mainWith filename = do
    input <- readFile filename
    let is = parseLine input
        r = runIProgram is
    putStrLn $ show r

parseLine :: String -> [Instr]
parseLine s = fmap parseWord $ words s
    where
        parseWord :: String -> Instr
        parseWord "Add" = IBinOp Add
        parseWord "Sub" = IBinOp Sub
        parseWord "Mul" = IBinOp Mul
        parseWord "Div" = IBinOp Div
        parseWord "Mod" = IBinOp Mod
        parseWord "Neg" = IUnOp Neg
        parseWord w     = IConst $ read w
```

Llistat 2: Instr1.hs

```
module Instr1 where
import Ops

type IProgram = [Instr]

data Instr = IConst Int
           | IBinOp BinOp
           | IUnOp UnOp
    deriving Show

runIProgram :: [Instr] -> Int
runIProgram is =
    let xs = runInstrs is []
    in runTop xs

type Stack = [Int]

runTop :: Stack -> Int
runTop (x : _) = x

runInstrs :: [Instr] -> Stack -> Stack
runInstrs is xs =
    foldl (flip runInstr1) xs is

runInstr1 :: Instr -> Stack -> Stack
runInstr1 (IConst x) xs =
    x : xs
runInstr1 (IBinOp op) (x2 : x1 : xs) =
```

```

doBinOp op x1 x2 : xs
runInstr1 (IUnOp op) (x1 : xs) =
doUnOp op x1 : xs

```

Llistat 3: Ops.hs

```

module Ops where

data BinOp = Add | Sub | Mul | Div | Mod
           | Eq | Le
  deriving (Show, Read, Eq, Enum, Bounded)

data UnOp = Neg
          | Not
  deriving (Show, Read, Eq, Enum, Bounded)

doBinOp :: BinOp -> Int -> Int -> Int
doBinOp Add = (+)
doBinOp Sub = (-)
doBinOp Mul = (*)
doBinOp Div = div
doBinOp Mod = mod

doBinOp Eq = \x y -> if x == y then 1 else 0
doBinOp Le = \x y -> if x <= y then 1 else 0

doUnOp :: UnOp -> Int -> Int
doUnOp Neg = negate

doUnOp Not = \x -> if x == 0 then 1 else 0

```

6.2 Màquina de pila simple (versió 2)

Llistat 4: mps2.hs

```

-- Tractament d'errors (amb el monad 'Maybe').

module Main where
import Instr2
import Ops

import System.Environment (getArgs)
import Text.Read -- exporta 'readMaybe :: (Read a) => String -> Maybe a'

main :: IO ()
main = do
  [filename] <- getArgs
  mainWith filename

mainWith :: String -> IO ()
mainWith filename = do -- Monad IO
  input <- readFile filename
  let r_err = do -- Monad Maybe
    is <- parseLine input
    runIProgram is
  case r_err of
    Just r -> putStrLn $ "Ok: " <> show r
    Nothing -> putStrLn "Error"

parseLine :: String -> Maybe [Instr]
parseLine s = traverseList parseWord $ words s
  where
    parseWord :: String -> Maybe Instr
    parseWord "Add" = pure $ IBinOp Add
    parseWord "Sub" = pure $ IBinOp Sub
    parseWord "Mul" = pure $ IBinOp Mul
    parseWord "Div" = pure $ IBinOp Div
    parseWord "Mod" = pure $ IBinOp Mod
    parseWord "Neg" = pure $ IUnOp Neg

```

```

        parseWord w      = IConst <$> readMaybe w

traverseList :: Applicative m => (a -> m b) -> [a] -> m [b]
traverseList ... =
    -- A completar per l'estudiant
    ...

```

Llistat 5: Instr2.hs

```

-- Tractament d'errors (amb el monad 'Maybe').

module Instr2 where
import Ops

import Data.Foldable
-- exporta
-- 'foldlM :: (Foldable t, Monad m) =>
--   (b -> a -> m b) -> b -> t a -> m b'

type IProgram = [Instr]

data Instr = IConst Int
           | IUnOp UnOp
           | IBinOp BinOp
           deriving Show

runIProgram :: IProgram -> Maybe Int
runIProgram is = do -- Monad Maybe
    xs <- runInstrs is []
    runTop xs

type Stack = [Int]

runTop :: Stack -> Maybe Int
runTop [x] = Just x
runTop _   = Nothing

runInstrs :: IProgram -> Stack -> Maybe Stack
runInstrs is xs =
    foldlM (flip runInstr1) xs is

runInstr1 :: Instr -> Stack -> Maybe Stack
runInstr1 (IConst x) xs =
    Just $ x : xs
runInstr1 (IUnOp op) (x1 : xs) =
    Just $ doUnOp op x1 : xs
runInstr1 (IBinOp op) (x2 : x1 : xs) =
    Just $ doBinOp op x1 x2 : xs
runInstr1 _ _ =
    Nothing

```

6.3 Màquina de pila simple (versió 3)

Llistat 6: mps3.hs

```

-- Tractament d'errors (amb el monad 'Either String').

module Main where
import Instr3
import Ops

import System.Environment (getArgs)
import Text.Read

main :: IO ()
main = do
    [filename] <- getArgs
    mainWith filename

mainWith :: String -> IO ()

```

```

mainWith filename = do
    input <- readFile filename
    -- A completar per l'estudiant

parseLine :: String -> Either String [Instr]
parseLine s =
    -- A completar per l'estudiant

```

Llistat 7: Instr3.hs

```

-- Tractament d'errors (amb el monad 'Either String').

module Instr3 where
import Ops

import Data.Foldable
-- exporta
-- 'foldlM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b'

type IProgram = [Instr]

data Instr = IConst Int
           | IUnOp UnOp
           | IBinOp BinOp
           deriving Show

runIProgram :: IProgram -> Either String Int
runIProgram is =
    error "runIProgram: A completar per l'estudiant"

type Stack = [Int]

{- A completar per l'estudiant:
    runTop :: Stack -> Either String Int
    runInstrs :: IProgram -> Stack -> Either String Stack
    runInstr1 :: Instr -> Stack -> Either String Stack
-}

```

6.4 Màquina de pila simple amb monad estat

Llistat 8: Instr1Estat.hs

```

module Instr1Estat where
import Ops

type IProgram = [Instr]

data Instr = IConst Int
           | IUnOp UnOp
           | IBinOp BinOp
           deriving Show

runIProgram :: IProgram -> Int
runIProgram is =
    let (_, xs) = runICodeM (runInstrs is) []
    in runTop xs

type Stack = [Int]

runTop :: Stack -> Int
runTop (x : _) = x

newtype ICodeM a = ICodeM { runICodeM :: Stack -> (a, Stack) }

instance Functor ICodeM where
    -- fmap :: (a -> b) -> ICodeM a -> ICodeM b
    fmap g m = error "ICodeM.fmap: A completar per l'estudiant"

instance Applicative ICodeM where

```

```

-- pure :: a -> ICodeM a
pure x = error "ICodeM.pure: A completar per l'estudiant"
-- (<*>) :: ICodeM (a -> b) -> ICodeM a -> ICodeM b
mf <*> mx = ICodeM $ \ xs ->
    let (f, xs') = runICodeM mf xs
        (x, xs'') = runICodeM mx xs'
    in (f x, xs'')

instance Monad ICodeM where
    -- (>=) :: ICodeM a -> (a -> ICodeM b) -> ICodeM b
    mx >= k = error "ICodeM.>=: A completar per l'estudiant"

runInstrs :: IProgram -> ICodeM ()
runInstrs is = mapM_ runInstr1 is

runInstr1 :: Instr -> ICodeM ()
runInstr1 = error "runInstr1: A completar per l'estudiant"
-- (usant push i pop)

push :: Int -> ICodeM ()
push x = ICodeM $ \ xs -> ((), x : xs)

pop :: ICodeM Int
pop = error "pop: A completar per l'estudiant"

```

6.5 Màquina d'expressions

Llistat 9: me.hs

```

import ESyntax
import Analitzador
import ECompile
import Instr3
import Ops

import System.Environment
import System.IO

main = do
    [filename] <- getArgs
    mainWith filename

mainWith :: String -> IO ()
mainWith filename = do
    input <- readFile filename
    analitza input

analitza :: String -> IO ()
analitza input = do
    let r = execAnalitzador (espaisEnBlanc *> exprP <*> fiTextA) input
    case r of
        Nothing -> putStrLn "Syntax error"
        Just (e, _) -> do
            print e
            tradueix e

tradueix :: Expr -> IO ()
tradueix e = do
    case compila e of
        Left err -> putStrLn $ "Error: " <> err
        Right is -> do
            print is
            executa is

executa :: IProgram -> IO ()
executa is =
    case runIProgram is of
        Right r -> putStrLn $ "Ok: " <> show r
        Left err -> putStrLn $ "Error: " <> err

```

Llistat 10: ESyntax.hs

```

module ESyntax
where
import Analitzador

import Control.Applicative
import Data.Char

data Expr = ELit Int
          | EApp String [Expr]
          deriving Show

exprP :: Analitzador Expr
exprP = error "exprP: A completar per l'estudiant"

atomP :: Analitzador Expr
atomP = error "atomP: A completar per l'estudiant"

-----

-- Aquests parsers (numero, simbol i ident) eliminen l'espai en blanc
-- que pugui haver-hi despres del text analitzat.

numero :: Analitzador Int
numero = error "numero: A completar per l'estudiant"
        -- usant eliminaEspais i enterPosA

simbol :: String -> Analitzador String
simbol = error "simbol: A completar per l'estudiant"
        -- usant eliminaEspais i cadenaA

ident :: Analitzador String
ident =
    eliminaEspais $
        (:) <$> complirA isAlpha <*> many (complirA isAlphaNum)

parens :: Analitzador a -> Analitzador a
parens p = error "parens: A completar per l'estudiant"

-- Aplica l'analitzador indicat i elimina l'espai en blanc
-- que pugui haver-hi despres del text analitzat.
eliminaEspais :: Analitzador a -> Analitzador a
eliminaEspais p = error "eliminaEspais: A completar per l'estudiant"

-- Multiples espais o comentaris
espaisEnBlanc :: Analitzador ()
espaisEnBlanc = error "espaisEnBlanc: A completar per l'estudiant"

comentari :: Analitzador ()
comentari =
    cadenaA "--" *> many (complirA (/= '\n')) *> complirA (== '\n') *> pure ()

```

Llistat 11: Analitzador.hs

```

module Analitzador
where
import Control.Applicative
import Data.Char

newtype Analitzador a =
    Analitzador { execAnalitzador :: String -> Maybe (a, String) }

complirA :: (Char -> Bool) -> Analitzador Char
complirA predicat = Analitzador f
    where
        f [] = Nothing          -- falla si l'entrada es buida

```

```

f (x:xs) =          -- Comprova si x compleix el predicat
    if predicat x then Just (x, xs) -- si es que si
                                -- retorna x i la resta de
                                -- l'entrada (es a dir, xs)
    else Nothing    -- altrament, falla

caracterA :: Char -> Analitzador Char
caracterA c = complirA (== c)

enterPosA :: Analitzador Int
enterPosA = Analitzador f
    where
        f xs = if null digits then Nothing
                else Just (read digits, elQueQueda)
        where (digits, elQueQueda) = span isDigit xs

fiTextA :: Analitzador ()
fiTextA = Analitzador f
    where
        f [] = pure ((), [])
        f _  = Nothing

instance Functor Analitzador where
    -- A completar per l'estudiant
    -- fmap :: (a -> b) -> Analitzador a -> Analitzador b

instance Applicative Analitzador where
    -- A completar per l'estudiant
    -- pure :: a -> Analitzador a
    -- (<*>) :: Analitzador (a -> b) -> Analitzador a -> Analitzador b
    (<*>) = error "<*>: A completar per l'estudiant"

instance Alternative Analitzador where
    -- A completar per l'estudiant
    -- empty :: Analitzador a
    -- (<|>) :: Analitzador a -> Analitzador a -> Analitzador a

cadenaA :: String -> Analitzador String
cadenaA = error "cadenaA: A completar per l'estudiant"

```

Llistat 12: ECompile.hs

```

module ECompile
where
import ESyntax
import Instr3
import Ops

import Control.Applicative

compila :: Expr -> Either String IProgram
compila (ELit x) =
    pure [IConst x]

compila (EApp name args) = do
    (np, i) <- case getOp name of
        Just p -> pure p
        Nothing -> Left $ "Undefined operator: " <> name
    if np == length args then pure ()
    else Left $ "Expected "<>show np<>" arguments"
    -- compila els arguments, obtenint les corresponents instruccions,
    -- i afegeix la instrucció 'i' al final
    error "compila: A completar per l'estudiant"

-- 'getOp name' obté 'Nothing' si 'name' no és una de les operacions definides ("
-- add", "sub", ...),
-- o obté 'Just (np, i)' si correspon a la instrucció 'i' amb 'np' arguments.
getOp :: String -> Maybe (Int, Instr)
getOp name = error "getOp: A completar per l'estudiant"
    -- usant el fet que Maybe es instancia d'Alternative

```

```
-- (useu binOps, unOps, fmap i (<|>))

binOps :: String -> Maybe BinOp
binOps "add" = Just Add
binOps "sub" = Just Sub
binOps "mul" = Just Mul
binOps "div" = Just Div
binOps "mod" = Just Mod
binOps _     = Nothing

unOps :: String -> Maybe UnOp
unOps "neg" = Just Neg
unOps _     = Nothing
```